SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA51

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation. CO (BL4, BL5)

Assessment Tool Weightage:	20%
QUESTIONS:	5
TOTAL MARKS:	25

Annexure A

CASE STUDY

Code of Conduct:

I **RITHISH KUMAR D** (Name / Reg No: **192565089**) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ***Rithish Kumar D***

Annexure A

Case Study

1. A healthcare system uses MD5 to store patient record hashes. A data breach reveals that attackers were able to generate collisions and manipulate records. Analyze how MD5's weaknesses contributed to this attack and recommend a secure hashing and integrity verification mechanism.

2. A banking application uses the Secure Hash Algorithm (SHA-256) for securing transactions, but performance issues arise during peak hours. Evaluate the trade-off between security and efficiency, and propose an optimized solution without compromising data integrity.

3. A government agency implements digital signatures for e-governance services. A fraudulent transaction is reported where a user denies signing a document. Analyze how digital signatures provide authentication and non-repudiation, and design a verification process to resolve the dispute.

4. In a corporate network using Kerberos authentication, a replay attack is suspected after unauthorized access is detected. Analyze how Kerberos prevents such attacks and evaluate what misconfigurations could have allowed this breach.

5. An enterprise relies on X.509 certificates for SSL/TLS communication. During an audit, it is discovered that expired and self-signed certificates are still trusted within the system. Evaluate the risks involved and propose a robust certificate lifecycle and trust management strategy.

Question 1

A healthcare system uses MD5 to store patient record hashes. A data breach reveals that attackers were able to generate collisions and manipulate records. Analyze how MD5's weaknesses contributed to this attack and recommend a secure hashing and integrity verification mechanism.

Answer:

1. Scenario Analysis

The hospital's record-management system relies on MD5 to fingerprint patient records so that any unauthorized change can, in theory, be detected by comparing a stored hash against a freshly computed one. During the breach, attackers did not simply steal data; they were able to modify patient records - diagnoses, prescriptions, billing details - and still produce a hash that matched the value already on file. This means the integrity-verification control itself was defeated, which is far more dangerous than a simple confidentiality breach because clinicians and auditors would have no reliable way of knowing which records had been tampered with.

2. Technical Analysis: Why MD5 Failed

MD5 produces only a 128-bit digest and uses a Merkle-Damgard construction that has been shown, since Wang et al.'s work in 2004 and subsequent refinements, to be vulnerable to practical collision attacks. A collision means two different inputs - in this case, the original record and a maliciously altered record - can be crafted to produce an identical MD5 hash. Chosen-prefix collision techniques (Stevens et al., 2007-2009) go further, allowing an attacker to choose two arbitrary starting record contents and still force a collision, which is exactly the scenario needed to make a falsified record look authentic. What used to require an infeasible 2^64 operations under a naive birthday attack can now be done in minutes on commodity hardware. Because the healthcare system used MD5 purely as an unkeyed checksum rather than a keyed authentication code, an attacker who could read or predict the storage format did not even need to compromise a secret key - only the hash algorithm's mathematical weakness - to forge a matching digest. In addition, MD5 hashes were apparently stored without any digital signature or access control binding them to a trusted origin, so once a matching hash was produced there was no secondary mechanism (such as a signed audit trail) to flag the change.

3. Recommended Secure Hashing and Integrity Mechanism

The healthcare provider should retire MD5 immediately and adopt SHA-256 or SHA-3-256 as the underlying hash function, since neither has any known practical collision or pre-image attack. However, replacing the algorithm alone is not sufficient; the integrity architecture itself should be strengthened as follows. First, record hashes should be computed as HMAC-SHA-256 using a secret key held in a Hardware Security Module (HSM), so that even if an attacker can read the stored hash, they cannot forge a new one without the key. Second, every record update should be digitally signed (RSA-2048/ECDSA P-256 with SHA-256) by the authorized clinician's credential, which provides both integrity and non-repudiation - the system can prove who made a change and detect any unauthorized change to the signed content. Third, a Merkle-tree or hash-chain structure should be used across the patient record database (similar to how blockchains and certificate-transparency logs work), so that tampering with any single record breaks a chain of hashes that spans many records, making silent, isolated forgery far harder. Fourth, all hash and signature verification events should be logged to a write-once, append-only audit log stored separately from the primary database, with periodic automated integrity sweeps that alert administrators the moment a mismatch is found. Finally, legacy MD5 hashes

already in the database should be migrated by re-hashing records under supervised, verified conditions, and MD5 should be disallowed at the application layer going forward, in line with HIPAA's requirement for reasonable and appropriate safeguards over electronic protected health information (ePHI).

4. Conclusion

This case illustrates a classic failure of using a cryptographically broken hash function for security-critical integrity checks. MD5's mathematically demonstrated collision weakness allowed attackers to defeat the very control meant to catch tampering. Moving to SHA-256/SHA-3 combined with HMAC keying, digital signatures for non-repudiation, and a tamper-evident chained log structure closes this gap and brings the system in line with modern, industry-accepted integrity-assurance practice for healthcare data.

Question 2

A banking application uses the Secure Hash Algorithm (SHA-256) for securing transactions, but performance issues arise during peak hours. Evaluate the trade-off between security and efficiency, and propose an optimized solution without compromising data integrity.

Answer:

1. Scenario Analysis

The bank hashes every transaction with SHA-256 to guarantee its integrity and, typically, to feed the digital-signature or MAC process that authenticates the transaction before it is committed. During peak hours - salary days, festival shopping periods, month-end settlements - transaction volume spikes sharply, and the cumulative CPU cost of computing millions of SHA-256 digests per hour becomes a bottleneck, increasing transaction latency and, in the worst case, causing timeouts or queued requests that frustrate customers.

2. Security-vs-Efficiency Trade-off

SHA-256 is deliberately more computationally expensive than legacy algorithms like MD5 or SHA-1 because its larger 256-bit state and more complex message schedule (64 rounds over 32-bit words) are what give it strong collision and pre-image resistance. Any attempt to reduce this cost by switching to a weaker or truncated hash function would directly undermine the integrity and authentication guarantees the bank depends on - in a regulated financial environment (PCI-DSS, RBI/central-bank guidelines), downgrading algorithm strength is not an acceptable trade-off, because the cost of a single successful integrity or authentication breach (fraudulent transactions, regulatory penalties, reputational damage) vastly outweighs the operational cost of extra CPU cycles. The correct trade-off, therefore, is not between security and speed at the algorithm level, but between infrastructure cost and throughput: the bank should scale and optimize how SHA-256 is computed and used, rather than weaken the algorithm.

3. Proposed Optimized Solution

A layered, non-compromising optimization strategy is recommended. First, hashing should be offloaded to dedicated Hardware Security Modules (HSMs) or CPUs with SHA extensions (Intel SHA-NI / ARMv8 Crypto Extensions), which compute SHA-256 several times faster than software-only implementations and simultaneously provide secure key storage for any associated HMAC or signing keys. Second, the transaction-processing tier should be horizontally scaled using stateless microservices behind a load balancer, so hashing work is distributed across many nodes during peak load rather than bottlenecking on a single server. Third, the bank can introduce asynchronous, pipelined processing: the customer-facing acknowledgement can be returned as soon as the transaction is queued and hashed at the ingress layer, while heavier downstream reconciliation hashing is processed in parallel batches, keeping perceived latency low without skipping any integrity checks. Fourth, for very high-frequency internal operations that are not directly regulated as the primary transaction-integrity control (e.g., internal caching, session-token validation, deduplication checks), the bank may evaluate BLAKE2b/BLAKE3, which are cryptographically strong, SHA-3-competition-derived designs that run faster than SHA-256 in software, while still using SHA-256 for the core, regulator-mandated transaction-integrity and signature workflows. Fifth, auto-scaling policies tied to real-time transaction-volume metrics should provision additional compute capacity automatically ahead of known peak windows (paydays, festivals), smoothing out load spikes proactively rather than reactively.

4. Conclusion

Security and efficiency need not be in direct conflict if the optimization happens at the infrastructure and architecture level rather than by weakening the cryptographic primitive. By combining hardware-accelerated SHA-256, horizontal scaling, asynchronous pipelines, and selective use of fast modern hash functions for non-critical paths, the bank can absorb peak-hour load while keeping SHA-256 as the trusted, unweakened foundation for transaction integrity.

Question 3

A government agency implements digital signatures for e-governance services. A fraudulent transaction is reported where a user denies signing a document. Analyze how digital signatures provide authentication and non-repudiation, and design a verification process to resolve the dispute.

Answer:

1. Scenario Analysis

A citizen used the government's e-governance portal to digitally sign a document (for example, a benefits application or a property transaction), and the signature was accepted as valid at the time. The citizen later disputes having signed it, and the agency must determine, with strong evidentiary confidence, whether the signature was genuinely produced by that citizen's private key or whether the dispute has merit (e.g., identity theft, key compromise, or a system flaw).

2. How Digital Signatures Provide Authentication and Non-repudiation

A digital signature is generated by hashing the document (e.g., with SHA-256) and then encrypting that hash with the signer's private key, typically issued as part of a Public Key Infrastructure (PKI) where a Certificate Authority (CA) has already verified the citizen's identity and bound it to a public/private key pair via an X.509 certificate. Authentication follows because only the holder of the private key could have produced a signature that successfully verifies under the corresponding public key; anyone can check this using the publicly available certificate. Non-repudiation follows from the same asymmetric property in reverse: because the private key is meant to be exclusively controlled by the signer (ideally stored in a smart card, USB token, or HSM protected by a PIN or biometric), the signer cannot later plausibly claim someone else produced the signature, provided the key was not compromised and proper identity vetting occurred at enrolment. This is fundamentally different from shared-secret schemes such as MACs, where either party holding the key could have produced the code, which is why MACs alone cannot provide non-repudiation.

3. Designing a Verification Process to Resolve the Dispute

To resolve the dispute the agency should follow a structured forensic-verification workflow. Step one is to retrieve the exact signed document and the accompanying signature value from the system of record, ensuring the retrieved copy has not itself been altered since signing (verified via its own stored hash). Step two is to retrieve the signer's X.509 certificate that was active at the time of signing and validate the full certificate chain up to the trusted government root CA, including checking that the certificate had not expired or been revoked at the signing time - this requires consulting historical Certificate Revocation List (CRL) or OCSP records rather than the certificate's present status. Step three is to cryptographically re-verify the signature: recompute the SHA-256 hash of the retrieved document and confirm it matches when decrypted with the certificate's public key. Step four is to check a trusted timestamp (RFC 3161 Timestamp Authority) that should have been embedded at signing time, which proves the signature existed at that specific moment and was not created retroactively. Step five is to examine supporting session evidence such as login logs, device fingerprints, IP address, OTP/2-factor authentication records, and any biometric or PIN-entry confirmation captured by the smart card or token middleware at the time of signing, since these corroborate that the citizen (or their device) was actively present. Step six is to investigate whether there is any evidence of private-key compromise - malware reports, lost/stolen token records, or unusual signing patterns - which would be the only legitimate basis for disowning an otherwise cryptographically valid signature. If all cryptographic and

procedural checks pass and no compromise evidence exists, the signature stands as valid proof of the citizen's action; if a break is found in the chain (e.g., a revoked certificate was still accepted, or a key-compromise report predates the signature), the agency has a technical basis to void the transaction and investigate a system or process failure instead.

4. Conclusion

Digital signatures give e-governance systems strong cryptographic authentication and non-repudiation, but that strength depends entirely on rigorous certificate validation, secure private-key custody, and reliable timestamping. A well-designed, step-by-step verification workflow - covering document integrity, certificate-chain validity at the time of signing, cryptographic re-verification, trusted timestamps, and corroborating session evidence - gives the agency a defensible, evidence-based way to resolve signature disputes fairly.

Question 4

In a corporate network using Kerberos authentication, a replay attack is suspected after unauthorized access is detected. Analyze how Kerberos prevents such attacks and evaluate what misconfigurations could have allowed this breach.

Answer:

1. Scenario Analysis

The organization's internal file and application servers rely on Kerberos, issued by an Active Directory Key Distribution Center (KDC), for single sign-on authentication. Security monitoring detected unauthorized access to a sensitive resource under a legitimate employee's identity at a time the employee states they were not active, which strongly suggests an attacker intercepted and re-used ('replayed') a previously issued ticket or authenticator rather than compromising the employee's password directly.

2. How Kerberos Is Designed to Prevent Replay Attacks

Kerberos incorporates several complementary defenses against replay. Every ticket (TGT or service ticket) carries a bounded validity window with explicit start and expiry timestamps, so a captured ticket becomes useless once that window elapses (typically hours for a TGT, minutes for some service tickets, configurable by policy). In addition to the ticket itself, the client sends a fresh 'authenticator' - a small structure encrypted with the session key and containing the current timestamp - with every request; the server decrypts it and rejects the request if the timestamp is not within an acceptable clock-skew window (5 minutes by default in most Kerberos implementations) or if it has already seen that exact authenticator before (this is why accurate, synchronized clocks across the domain, via NTP, are a hard requirement, not an optional nicety). Servers also maintain a short-lived replay cache of recently seen authenticators within the valid clock-skew window, so that even a timestamp-valid authenticator cannot be reused a second time. Finally, mutual authentication - the server can prove back to the client that it also holds the shared session key - prevents an attacker from impersonating the server to harvest credentials in the first place.

3. Evaluating Likely Misconfigurations Behind the Breach

Given that unauthorized access still occurred, several plausible misconfigurations should be investigated. First, excessive clock skew tolerance: if administrators widened the allowed time-skew window (for example, to accommodate poorly synchronized branch office servers) well beyond the recommended 5 minutes, a captured authenticator remains usable for a much longer window than intended. Second, disabled or missing replay-cache enforcement on a service host - some legacy or third-party services do not properly implement the replay cache, silently accepting a repeated authenticator. Third, excessively long ticket lifetimes configured at the KDC (e.g., extending default TGT renewal well beyond the domain default) that give an attacker who intercepts a ticket via network sniffing or a compromised endpoint a far larger window to replay it. Fourth, weak encryption types still enabled on accounts - if RC4-HMAC (a legacy, weaker cipher) is still permitted alongside AES256-CTS-HMAC-SHA1, an attacker may find it easier to crack a captured ticket or forge session-key material, enabling more powerful attacks such as 'pass-the-ticket' rather than a pure timing-based replay. Fifth, Kerberos pre-authentication disabled on one or more accounts, which removes an important barrier against offline attacks that could lead to ticket or key compromise in the first place. Sixth, insufficient monitoring: without SIEM alerting on anomalies such as the same ticket

or authenticator being used from two different source IP addresses in quick succession, or repeated AS-REQ/TGS-REQ failures, a replay can go undetected long enough to cause damage. Finally, a stale or unrotated KRBTGT account password increases the blast radius if any ticket-forging technique (e.g., a golden-ticket style attack) is combined with the replay, since forged tickets remain valid until that password is changed.

4. Conclusion

Kerberos is designed with strong timestamp-, lifetime-, and cache-based defenses against replay attacks, but those defenses are only as strong as their configuration. This breach most likely stems from one or more operational gaps - loose clock-skew tolerance, missing replay-cache enforcement, over-long ticket lifetimes, legacy weak encryption types, or insufficient monitoring - rather than a flaw in the Kerberos protocol itself. Tightening these configurations, enforcing AES-only encryption, enabling pre-authentication domain-wide, synchronizing clocks via NTP, and adding anomaly-based monitoring would close the gap.

Question 5

An enterprise relies on X.509 certificates for SSL/TLS communication. During an audit, it is discovered that expired and self-signed certificates are still trusted within the system. Evaluate the risks involved and propose a robust certificate lifecycle and trust management strategy.

Answer:

1. Scenario Analysis

An internal security audit of the enterprise's SSL/TLS infrastructure found two concerning issues: several endpoints are still presenting certificates that have already passed their 'Not After' expiry date, and other internal services are trusted purely on the basis of self-signed certificates that were never issued by a recognized Certificate Authority (CA). Both conditions mean the organization's TLS trust model no longer reliably guarantees the identity of the systems it is communicating with.

2. Risk Evaluation

Expired certificates are risky because certificate validity periods exist specifically to bound the time during which the CA vouches for the binding between an identity and a public key; an expired certificate may correspond to a key whose custody, server configuration, or organizational ownership has since changed, so continuing to trust it removes a key safety check. Expired certificates are also frequently associated with outdated TLS configurations - older cipher suites, weaker key sizes, or unpatched servers - increasing exposure to downgrade and cipher-suite attacks. Self-signed certificates being trusted enterprise-wide is a more serious risk: because there is no independent, trusted third party vouching for the identity in the certificate, an attacker who can position themselves on the network (e.g., via ARP spoofing, DNS poisoning, or a rogue access point) can present their own self-signed certificate and successfully impersonate a legitimate internal service, enabling a man-in-the-middle attack that intercepts or alters supposedly secure traffic. If users or systems have been trained (through habitual security-warning bypass) to click through browser or client warnings for these certificates, that same conditioned behavior can be exploited against a genuinely malicious certificate presented later. Beyond the direct technical risk, this state of affairs typically represents a compliance failure - standards such as PCI-DSS, ISO 27001, and most regulatory data-protection frameworks require valid, properly-issued and monitored certificates for systems handling sensitive data, so an audit finding of this kind can trigger regulatory penalties and loss of certification.

3. Proposed Certificate Lifecycle and Trust Management Strategy

The enterprise should implement a comprehensive Certificate Lifecycle Management (CLM) program built on five pillars. First, discovery and inventory: deploy automated network scanning tools to build and continuously maintain a complete inventory of every certificate in use across the enterprise, including internal, external, and third-party endpoints, since certificates that are not tracked cannot be managed. Second, governance and PKI hierarchy: replace ad-hoc self-signed certificates with an internally operated, properly rooted PKI (an internal Root CA and issuing intermediate CAs) so that all internal services present certificates that are chained to a trust anchor the organization's devices are explicitly configured to trust - eliminating the security theatre of unmanaged self-signed certificates while still avoiding the cost of public CA issuance for purely internal endpoints. Third, automation: adopt automated issuance and renewal using the ACME protocol or enterprise certificate-management platforms (for example, HashiCorp Vault PKI or a commercial CLM tool), enforcing conservative maximum validity periods aligned with CA/Browser Forum baselines (currently 398 days for public

TLS certificates, and even shorter internally where feasible) so that certificates renew automatically well before expiry rather than relying on manual tracking. Fourth, active monitoring and revocation: implement real-time expiry alerting (30/15/7-day warnings) tied to an owner/team for each certificate, enable OCSP stapling or up-to-date CRL checking so revoked certificates are rejected immediately, and integrate certificate health checks into the existing SIEM/monitoring dashboards. Fifth, periodic audit and enforcement: schedule recurring audits (at least quarterly) to confirm no expired or unauthorized self-signed certificates remain trusted, enforce strict trust-store policy on all managed devices (removing ad-hoc trusted root entries added outside the change-management process), and where mutual TLS or certificate pinning is appropriate for high-value internal services, adopt it to add a further layer of defense beyond chain validation alone.

4. Conclusion

Expired and self-signed certificates being trusted in production undermines the entire purpose of TLS - verifying the identity of the party you are communicating with - and exposes the enterprise to man-in-the-middle attacks and compliance failures. A structured Certificate Lifecycle Management program combining full inventory, a properly governed internal PKI, automated issuance/renewal, real-time monitoring, and regular audits closes this gap and restores a trustworthy TLS posture.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand